

separate tab pages. For instance, the command below instructs Vim to open each file in a new tab page.

```
[bash]$ vim -p file1.txt file2.txt
```

If the number of tab pages is greater than the number of files provided at the command line, then the remaining tab pages open empty buffers. For instance, the command below opens a total of four tab pages, where the first two show *file1.txt* and *file2.txt*, respectively, while the remaining two show empty buffers.

```
[bash] vim -p4 file1.txt file2.txt
```

What if we want to open a tab page in the middle of our work? Don't worry about it. There is no need to close the current Vim session. Vim developers have already thought about this. There are two more commands, using which we can open tab pages in a running instance of Vim. We can use either of the following three commands—*:tabnew*, *:tabedit* or *:tabe*. The syntax for each command is given below:

```
: [N]tabnew [filename]
: [N]tabe [filename]
: [N]tabedit [filename]
```

If you omit the file name with the above commands then Vim opens empty tab pages; otherwise, it loads the file into a new tab page. If 'N' is omitted, the new tab page appears after the current one; otherwise, Vim opens a new tab page after the Nth tab page.

Tab pages are really awesome. But the question is: how many tab pages can we open? By default, we can open up to 10 tab pages, but this behaviour can be controlled by setting the *tabpagemax* option. For instance, to set an upper cap of 20 tab pages, add the following line into *~/.vimrc*:

```
set tabpagemax=20
```

If the number of tabs allowed by *tabpagemax* is exceeded, Vim will simply open the maximum number of tabs, and the other files will be opened but not displayed. You can edit the remaining files by using the *:next* or *:last* commands to move to the files that are not displayed in the tab pages. Please note that this setting is applicable only for the '-p' option; you can open as many tab pages as you like from a running instance of Vim.

In addition to this, Vim also provides the *tabfind* or *tabf* command, which finds the file in the current path and opens it in a new tab page. It uses Vim's *path* option to determine which directories should be searched when opening the specified file. To determine the value of the current path, just execute the *:set path* command. On my system, its value is as follows:

```
path=.,/usr/include,.
```

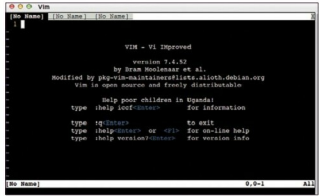


Figure 1: Tab pages

The above *path* instructs Vim to look in the directory containing the current file (*period()*), then the */usr/include* directory and, finally, the current directory (the empty text between two commas). To append an additional search path, execute the following command:

```
:set path += /etc
```

The steps below demonstrate how to use the *tabfind* command to find and open files in a new tab page:

```
[bash]$ cd /tmp/test      # Deliberately switch to the empty
                           # directory.
[bash]$ ls                # Verify it is empty.
[bash]$ vim               # Open instance of vim and execute
                           # the command below from vim
:tabfind malloc.h
```

Though the */tmp/test* directory doesn't contain the *malloc.h* file, Vim opens it by searching for it in the */usr/include* path. You can also use regular expressions with *tabfind*. Just as in the *:tabedit* command, when 'N' is omitted with the *:tabfind* command, a new tab page appears after the current one; otherwise, Vim opens a new tab page after the Nth tab page.

So far, we have seen several ways to open tab pages. Now, let us quickly go through the steps to close the tab pages. To close tab pages, we can use any of the following commands according to our requirements:

```
:tabclose
:tabclose [N]
:tabonly
```

As the name suggests, the *:tabclose* command closes the current tab page. But this command may fail, if the current tab page is the last tab page or the in-memory buffer is modified but not yet written to the disk.